

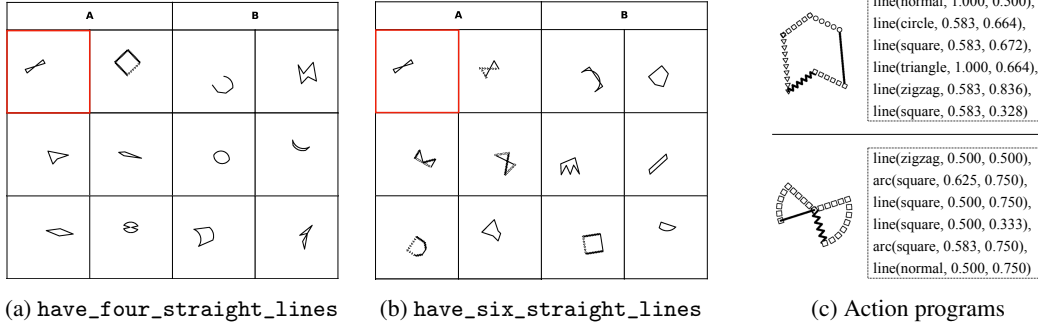


Figure 2: (a-b) An illustration of the context-dependent perception in BONGARD-LOGO, where (a) the concept is `have_four_straight_lines`, (b) the concept is `have_six_straight_lines`. The same shape pattern (highlighted in red) has fundamentally opposite interpretations depending on the context: Whether the line segments are seen as continuous when they intersect with each other. (c) Two exemplar shapes and the action programs that generate them procedurally, where each base action (`line` or `arc`) has three arguments (from left to right): *moving type*, *moving length* and *moving angle*. Note that there are five moving types, including `normal`, `zigzag`, `triangle`, `circle`, and `square`, each of which denotes how the line or arc is drawn. Also, both values of moving length and moving angle have been normalized into $[0, 1]$, respectively.

purpose of these problems is to test the ability of abstract concept discovery and reasoning. There are 25 abstract attributes in total, including `symmetric`, `convex`, `necked`, and so on. Each abstract attribute is shared by many different shapes while each shape is associated with multiple attributes. Due to the increased challenge of identifying the abstract concept, we randomly sample 20 different problems for each abstract shape concept. Figure 1c shows an example of abstract shape problems, where the underlying concept is `convex`. Similarly, the stroke types (i.e., zigzags or lines of circles, etc.) do not impact the convexity. Large variations in all convex and concave shapes ensure that the model does not simply memorize some finite shape templates but instead is forced to understand the gist of the `convex` concept.

2.2 Properties of the BONGARD-LOGO Problems

In the BONGARD-LOGO problems, we do not distinguish concepts by the shape size, orientation and position, or relative distance of two shapes. Furthermore, the Bongard-style reasoning needs the visual concept to be inferred from its context with a few examples. All these together demand a perception that is both rotation-invariant and scale-invariant, and more importantly, a new learning paradigm different from traditional image recognition methods. In the next, we mainly focus on three core characteristics of human cognition captured by BONGARD-LOGO, and use intuitive examples to explain them: 1) context-dependent perception, 2) analogy-making perception, and 3) perception with a few samples but of infinite vocabulary.

Context-dependent perception The very same geometrical arrangement may have fundamentally different representations for each context on which it arises. For example, the underlying concept in the task of Figure 2a is `have_four_straight_lines`. Straight line segments must be seen as continuous, even when they intersect with another line segment. This does not happen, in the task of Figure 2b, where intersections have to split the straight lines to fulfill the underlying concept `have_six_straight_lines`. Another representative example is the hierarchy in concept learning. An equilateral triangle can either be interpreted as `equilateral_triangle` or as `convex`, which completely depends on what the remaining positive images and negative images are.

This property is compliant with the so-called “*one object, many views*” in human cognition [10, 26], where the existence and meaning of an object concept depend on the human understanding. Inspired by the original BPs, our benchmark contains a large number of the above examples that require context-dependent perception: $P(X, C)$, where C is the context to which an object X belongs. It does not make sense to infer the concept from X alone, as there is a need for contextual information that lies outside of X . By contrast, most current pattern recognition models are implicitly *context-free*. For example, the image classification models take a single cat image and output a cat label, without referring to any other image as context. They assume that there exist objects in reality outside of any understanding, and the goal is to find the unique correct description [15]. Therefore, the perception

in the conventional context-free models is a one-to-one mapping from any object X to its unique one representation $P(X)$. This may account for the inefficacy of current pattern recognition models, founded on the premise of context-free perception, in our benchmark.

Analogy-making perception When humans make an analogy, we interpret an object in terms of another. In such a sense, representations can be traded off – we may interpret a zigzag as a straight line, or a set of triangles as an arc, or just about anything as another thing [15, 27]. When we trade off the zigzag for a straight line, we not only interpret it as a straight line, but also project onto it all the properties of a straight line, such as straightness. Though zigzags can never be straight, one could easily imagine a zigzagged line and a zigzagged arc [27], as shown in Figure 1.

Many problems in our benchmark require an analogy-making perception. Figure 1b shows an example where the underlying concept includes trapezoid, even if some trapezoids have no straight lines. Instead, they are a set of circles or zigzags that form a conceptual shape of trapezoid. Importantly, this may not be just an issue of noise or nuisance, because when we tackle the task, we trade off some concepts for other concepts. On one hand, we have circles or zigzags as a meaningful structure for the underlying concept of free-form problems. That is, we strictly distinguish a circle shape from a triangle shape, or a zigzagged line from a straight line in free-form problems. On the other hand, we trade off a set of triangles or zigzags for a trapezoid (Figure 1b) in the basic shape problems, and trade off a set of circles for a convex concept (Figure 1c) in the abstract attribute problems. A model with analogy-making perception will precisely know when a representation is crucial for the concept and when it has to be traded off for other concept.

Perception with a few examples but infinite vocabulary Unlike standard few-shot image classification benchmarks [4, 20, 28], there is no finite set of categories to name or standard geometrical arrangements to describe in BONGARD-LOGO. Similar to original BPs, our problem set is not just about dealing with triangles, circles, squares, or other easily categorizable shape patterns. Particularly in many free-form shape problems, it would be a formidable task to explain their content to others in a context-free manner, if they have not seen the problems before. For example, Figure 1a shows a representative example of free-form problems, where we humans cannot provide the precise name of the free-form shape, as a concept, but we are still able to easily recognize the concepts by observing the strokes, and then make the right decisions on classifying novel test images [29]. As each free-form shape is an arbitrary composition of randomly sampled basic stroke structures. The space of all possible combinations makes the shape vocabulary size infinite.

The infinite vocabulary forbids a few-shot learner from memorizing geometrical arrangements in a dataset rather than developing an ability to conceptualize. This property is consistent with practical observations of human visual cognition that infer concepts from a few examples and generalize them to vastly different situations [3, 25].

2.3 Problem Generation with Action-Oriented Language

A distinctive feature of the original BPs is that a visual concept can be implicitly and concisely communicated by a comparison between two sets of image examples. It requires careful construction of the image sets $\mathcal{A}$ and $\mathcal{B}$ such that the concept can be identified with clarity. M. M. Bongard manually designed the original set of one hundred problems to convey the concepts he had in mind. However, this manual generation process is not scalable. We procedurally generated our shapes in the LOGO language [16], where a so-called "turtle" moves under a set of procedural action commands and its trajectory produces vector graphics. For each shape, the corresponding procedural action commands form its ground-truth *action program*.

The use of action programs to generate shapes has several benefits: First, with action programs, we can easily generate arbitrary shapes and precisely control the shape variation in a human-interpretable way. For example, in the generation of free-form shapes, we randomly perturb one command in the ground-truth action programs to construct a similar-looking and challenging negative set. Second, the ground-truth action programs provide useful supervision in guiding symbolic reasoning in the action space. It has a great potential of promoting future methods that use the symbolic stimuli, such as neuro-symbolic AI. Note that there exists a "*one-to-many*" relationship between the action program and shape pattern: an action program *uniquely* determines a shape pattern while the same shape pattern may have *multiple* correct action programs that can generate it. Please see Appendix C for our preliminary results of incorporating symbolic information into neural networks for better performance. More importantly, although our benchmark has an infinite vocabulary of shape patterns,

the vocabulary of base actions are of relatively small size, making the concepts compositional in the action space and much easier to generate.

To simplify the process, we use only two classes of base actions: *line* and *arc*. As each base action has three arguments: *moving type*, *moving length* and *moving angle*, an action is depicted by a function: `[action_name]([moving_type], [moving_length], [moving_angle])`, as shown in Figure 2c. Specifically for the above arguments, there are five moving types, including *normal*, *zigzag*, *triangle*, *circle*, and *square*. For instance, *normal* means the line or arc is perfectly straight or curved, *zigzag* means the line or arc is of the zigzagged-style and *triangle* means the line or arc is formed by a set of triangles. Besides, both moving length and moving angle are normalized into the range of $[0, 1]$, where 0 and 1 correspond to the minimal and maximal values, respectively. Depending on varying lengths of action programs, different combinations of these base actions form visually distinct shapes. Our benchmark contains different types of shapes, as discussed in Section 2.1, each of which requires a separate procedure in the LOGO language to generate, and we leave generation details in Appendix A.

We have open-sourced the procedural generation code and data of BONGARD-LOGO in the following GitHub repository: <https://github.com/NVlabs/Bongard-LOGO>.

3 Experiments

3.1 Methods

We consider several state-of-the-art (SOTA) approaches, and test how they behave in BONGARD-LOGO, which demands human cognition abilities. First, as each task in BONGARD-LOGO can be cast as a *two-way six-shot* few-shot classification problem, where meta-learning has been a standard framework [30, 4], we first introduce the following meta-learning methods, with each being SOTA in different (i.e., memory-based, metric-based, optimization-based) meta-learning categories: 1) *SNAIL* [19], a memory-based method; 2) *ProtoNet* [20], a metric-based method; 3) *MetaOptNet* [21] and *ANIL* [22], two optimization-based methods. As the Meta-Baseline [23] is a new competitive baseline in many few-shot classification tasks, we consider its two variants: 1) *Meta-Baseline-SC*, where we meta-train the model from scratch, and 2) *Meta-Baseline-MoCo*, where first use an unsupervised contrastive learning method – MoCo [31] to pre-train the backbone network and then apply meta-training. Besides, we consider two non-meta-learning baselines for comparison. One is called *WReN-Bongard*, a variant of WReN [17] that was originally designed to encourage reasoning in the Raven-style Progressive Matrices (RPMs) [32]. Another one is a convolutional neural network (CNN) baseline, by casting the task into a conventional binary image classification problem, which we call *CNN-Baseline*. Please see Appendix B for the detailed descriptions of different models.

3.2 Benchmarking on BONGARD-LOGO

We split the 12,000 problems in BONGARD-LOGO into the disjoint train/validation/test sets, consisting of 9300, 900, and 1800 problems respectively. In the training set and validation set, we uniformly sample problems from three problem types in Section 2.1. To dissect the performances of the models in different problem types and levels of generalization, we use the following four test set splits:

Free-form shape test set This test set (FF) includes 600 free-form shape problems. To evaluate the capability of these models in *extrapolation* towards more complex free-form shape concepts than trained concepts, the shape patterns in this test set are generated with "one longer" action programs than the longest programs used for generating the training set. Note that we consider the test set of "one longer" action programs as a basic case for extrapolation. Since this task has been shown difficult for current methods (as shown in Table 1), we did not include more challenging setups.

Basic shape test set This test set (BA) includes 480 basic shape problems. We randomly sample 480 basic shape problems in which the concept corresponds to recognizing a composition of two shapes. As each basic shape concept only appears once in our benchmark, we ensure that the basic shape test set shares no common concept with the training set. This test set is designed to examine a model’s ability to generalize towards novel *composition* of basic shape concepts.

Combinatorial abstract shape test set This test set (CM) includes 400 abstract shape problems. We randomly sample 20 novel pairwise combinations of two abstract attributes, each with 20 problems. All the single attributes in this test set have been observed individually in the training set. However,